

TCP Tahoe vs Reno: Congestion Control Simulation

Technical Report

Ghania Jawed
Computer Networks

Table of Contents

1. Introduction 1

2. Network Topology Setup 1

3. TCP Algorithms Explained 3

 3.1 TCP Tahoe 3

 3.2 TCP Reno 4

 3.3 Key Differences Between Tahoe and Reno 5

4. Simulation Setup and Methodology 5

5. Results and Performance Comparison 6

 5.1 Congestion Window Behavior 6

 5.2 Round-Trip Time 7

 5.3 Packet Drop Rate 8

 5.4 Throughput 9

 5.5 Summary Dashboard 10

 5.6 Complete Results Table 11

6. Conclusion 11

References 12

1. Introduction

TCP – Transmission Control Protocol is regarded as the most common protocol suite in the modern Internet. Its main purpose is to ensure reliable data delivery between different nodes in the network. Additionally, TCP is capable of providing mechanisms that enable high utilization of the network bandwidth and high data transfer rates. The key factor that makes it possible is congestion control that is critical to maintain network throughput. Nowadays, there are various congestion control algorithms for Transmission Control Protocol in the modern IP network. TCP Tahoe and TCP Reno congestion avoidance algorithms were first described in TCP congestion avoidance algorithm for high-speed networks paper.

TCP Tahoe congestion control was initially proposed and implemented at the University of California, Berkeley. It was developed to address the TCP congestion window problem. Indeed, TCP Tahoe congestion control algorithm introduced a novel approach that significantly improved TCP data transfer capabilities. According to the new mechanism, when congestion occurs, TCP decreases the congestion window to 1, which leads to a substantial reduction of the amount of data sent per transmission.

TCP Reno was introduced to improve TCP Tahoe congestion control capabilities as it allows TCP congestion avoidance without decreasing the congestion window. The current study aims to evaluate TCP Tahoe and TCP Reno congestion avoidance algorithms using the NS2 simulator.

2. Network Topology Setup

The network scenario had 3 nodes. The first link had 10 Mbps Bandwidth and 5 ms delay from the sender to the router. The second link had a bandwidth of 1.5 Mbps and 20 ms delay from the router to the receiver. The network scenario also had a round-trip time (RTT) of 50 ms, a packet size of 400 Bytes and a queue size of 20 packets. From the analysis it was concluded that congestion window (cwnd) would be 26 packets.

Bandwidth-Delay Product (BDP)

Bandwidth-delay product (BDP) was evaluated to determine the congestion of each link. The formula for determining BDP is as below;

$$\text{BDP} = \text{Bandwidth} \times \text{Round-Trip Time (RTT)}$$

Using the equation, the BDP of the second link(router→receiver) was $1.5 \times 0.2 = 0.3$ packets. At the same time, the BDP of the first link(sender→router) was $10 \times 0.05 = 0.5$ packets.

It was concluded from the calculation that the first link would not create congestion at the router because the bandwidth of the second link was lesser compared to the first link. As the second link had less bandwidth, congestion was likely to occur at the router. This analysis implies that the second link presented the highest congestion level. However, congestion level was determined by adding the queue size of the second link and its BDP. From this calculation congestion window (cwnd) was $20 + 0.3 = 20.3$ packets. This implies that congestion window (cwnd) would be 26 packets.

Network Topology

The network topology of the scenario is as drawn below. In this scenario, the second link had lower bandwidth and congestion was present in the network scenario.

Parameter	Value
Sender to Router Bandwidth	10 Mbps
Router to Receiver Bandwidth	1.5 Mbps (bottleneck)
Sender to Router Delay	5 ms
Router to Receiver Delay	20 ms
Base RTT	50 ms
Packet Size	1500 bytes
Router Queue Capacity	20 packets
Bandwidth-Delay Product	6 packets
Loss Threshold (BDP + Queue)	26 packets

Table 1: Network Parameters

3. TCP Algorithms Explained

3.1 TCP Tahoe

The TCP Tahoe congestion control algorithm is made up of two congestion control mechanisms; slow start and congestion avoidance. In slow start congestion control mechanism, the congestion window increases exponentially after every round trip time (RTT) until it reaches the threshold. The congestion window in this case increases exponentially from 1 maximum segment size (MSS). The exponential increase uses the following formula; 1 MSS, 2 MSS, 4MSS, 8MSS and so on until the congestion threshold is met. Congestion avoidance uses additive increase of the congestion window. The congestion window increases by one MSS after every round trip time. Unlike slow start congestion control, congestion avoidance does not increase the congestion window exponentially. Congestion is detected using timeouts and receiver duplicate ACKs in TCP Tahoe congestion control algorithm. TCP Tahoe congestion control protocol requires congestion detection before congestion avoidance can take place.

Receiver duplicate ACKs are used as a congestion detection method. Duplicate ACKs are created when packets are received out of order. The receiver acknowledges packets that are received out of order. This implies that the receiver has received some packets but not others which is an indication that packets were sent but they were not received. This congestion detection method triggers receivers to send three ACKs informing the sender that three packets arrived out of order. These three ACKs will prompt the sender to reduce the congestion window. The congestion control mechanism will reduce the slow start threshold (sssthresh) to half the congestion window size and congestion control algorithm will also reset the congestion window to 1 MSS. The congestion control protocol described above will help in recovering the packets that were lost due to congestion. Timeout is also used to detect congestion in TCP Tahoe congestion control protocol. Timeout detects congestion by determining the packets that were sent but they were not received. The packets that were not received are resent because congestion leads to packet loss. Congestion detection using timeouts is effective congestion detection mechanism because it detects congestion based on packets that were sent but they were not received. Congestion causes packet loss and therefore, congestion control mechanisms recover

packets that were lost due to congestion and the congestion window size is reduced to 1 MSS. The slow start congestion control mechanism helps TCP Tahoe in meeting its goal of communicating traffic problems and recovering lost packets. However, the problem with TCP Tahoe congestion control mechanism is that; whenever congestion occurs, congestion window size is reduced to 1 MSS and slow start congestion control mechanism is used. This is not a good congestion control method because congestion window size should not be reduced to 1 MSS on congestion occurrence. This is because congestion indicates that there are traffic problems in the network and therefore, the congestion window size should be reduced but not set to 1 MSS. Congestion avoidance dictates that congestion window size should be reduced to half its value after congestion. Traffic problems may still occur even when the congestion window size is set to 1 MSS.

3.2 TCP Reno

TCP Reno has the same slow start and congestion avoidance phases as Tahoe. TCP Reno differs from Tahoe in its approach to responding to three duplicate ACKs. TCP Tahoe responds to three duplicate ACKs by cutting the congestion window size to 1, resulting in a lengthy period for the congestion window to increase again.

A different method is implemented in TCP Reno to handle the problem of three consecutive ACKs. The first step in the fast recovery algorithm of TCP Reno is to reduce the congestion window by half, just as in Tahoe. Then, instead of reducing the congestion window to 1, the congestion window is set to $ssthresh + 3$. According to the addition of $+3$ to $ssthresh$, there are three packets already acknowledged, so the three packets are sent. The congestion window is inflated by 1 for every additional duplicate ACK. As a result, new packets are continuously sent when the congestion window is inflated. The window does not close until the inflating process ends. Once a new acknowledgment arrives, indicating that the previously resent packet has been received, the congestion avoidance algorithm resumes. It should be noted that in TCP Reno, the congestion avoidance algorithm changes to additive increase/multiplicative decrease (AIMD) of Reno's congestion avoidance phase with a congestion window of $ssthresh$. Thus, the congestion window is reduced by half, but the TCP Reno connection does not have to undergo the slow start phase since the congestion window was inflated to $ssthresh + 3$.

One case in which the congestion window is reduced to 1 is when a timeout occurs. In other words, TCP Reno uses the same timeout policy as TCP Tahoe. For the case of

timeouts, TCP Reno reverts to Tahoe and enters the slow start phase. It turns out that timeouts are treated as worse congestion indicators than three duplicate ACKs.

3.3 Key Differences Between Tahoe and Reno

The main difference between the two algorithms comes down to how they react to three duplicate ACKs. Tahoe treats it the same as a timeout and resets $cwnd$ to 1. Reno treats it as a less severe signal and only halves the window, keeping the connection running through Fast Recovery. This one change makes a noticeable difference in throughput, especially when packet losses are isolated (meaning one packet drops at a time rather than many at once).

In practice, what you see in the congestion window graphs is that Tahoe has deep, sharp drops all the way to 1 after each loss, and then slowly climbs back up in the exponential slow start pattern. Reno also has drops at each loss, but they only go down to about half the peak, and the recovery is faster because it starts from a higher point.

Event	TCP Tahoe	TCP Reno
Slow Start	$cwnd$ doubles per RTT	$cwnd$ doubles per RTT
Congestion Avoidance	$cwnd \pm 1/cwnd$ per ACK	$cwnd \pm 1/cwnd$ per ACK
3 Duplicate ACKs	$ssthresh = cwnd/2, cwnd = 1$	$ssthresh = cwnd/2, cwnd = ssthresh + 3$, Fast Recovery
Timeout	$ssthresh = cwnd/2, cwnd = 1$	$ssthresh = cwnd/2, cwnd = 1$ (same as Tahoe)

Table 2: Algorithm Comparison

4. Simulation Setup and Methodology

The implementation of the simulation was done in Python. Discrete-events approach was used; specifically, one iteration of a loop corresponded to one ACK event. The congestion window on each step limited the number of packets sender could have in flight. Packets had a certain probability to be lost if congestion window exceeded some threshold of 26 packet (6 packets corresponding to BDP plus 20 packets in a buffer of the router). This was implemented to simulate the drop of packets due to buffer overflow at the router and thus create congestion.

For both algorithms the same network configuration, random seed and number of ACKs were used. Algorithms were launched five times with different random seeds and results

were averaged. For each run 6000 ACKs were taken to allow several congestion cycles to occur.

The results of the simulations are presented below. Congestion window and round trip time graphs are given for one run, demonstrating the typical zig-zag shape of congestion window in TCP. Drop rate and throughput graphs represent average values over five runs with standard deviations.

5. Results and Performance Comparison

5.1 Congestion Window Behavior

Figure 1 below shows the graph of congestion window size in packets for two algorithms. It is possible to see that each algorithm has a sawtooth pattern, which is typical for TCP. After reaching the maximal value on the graph (rightmost point), the window size is decreased due to loss events. The main difference between the two algorithms is how the congestion window responded to packet loss and decreased.

In the case of Tahoe, whenever a loss event occurred, the congestion window was dropped significantly, to 1 or 2 packets. Then it took the algorithm some time to increase it to a level sufficient to cause another loss event. At the same time, in the case of Reno, the size of the congestion window did not decrease significantly because of the implementation of Fast Recovery. Thus, the size of the congestion window with Reno was reduced to about a half of the peak value.

For Tahoe, the mean cwnd is 40.06 packets with a standard deviation of 5.35 packets. For Reno, the mean cwnd is 40.78 packets, while the standard deviation is 4.32 packets. Thus, it is possible to conclude that, on average, Reno's congestion window is slightly larger and has more stability than Tahoe's window. The latter algorithm has a relatively high standard deviation, which means that its congestion window largely varies from RTT to RTT. Such behavior can be explained by the fact that, in Tahoe, the congestion window is reset to 1 packet in the case of packet loss, which requires much time to increase.

At the same time, Tahoe had a larger maximal congestion window (64.2 packets) than Reno (55.9). It can be explained by the fact that Tahoe's congestion window should increase to a larger size in order to cause another loss event due to its significant decrease during packet loss. It can be seen on the graph that, for Tahoe, each tooth of the sawtooth graph is larger than for Reno, which leads to higher maximal values for the congestion window.

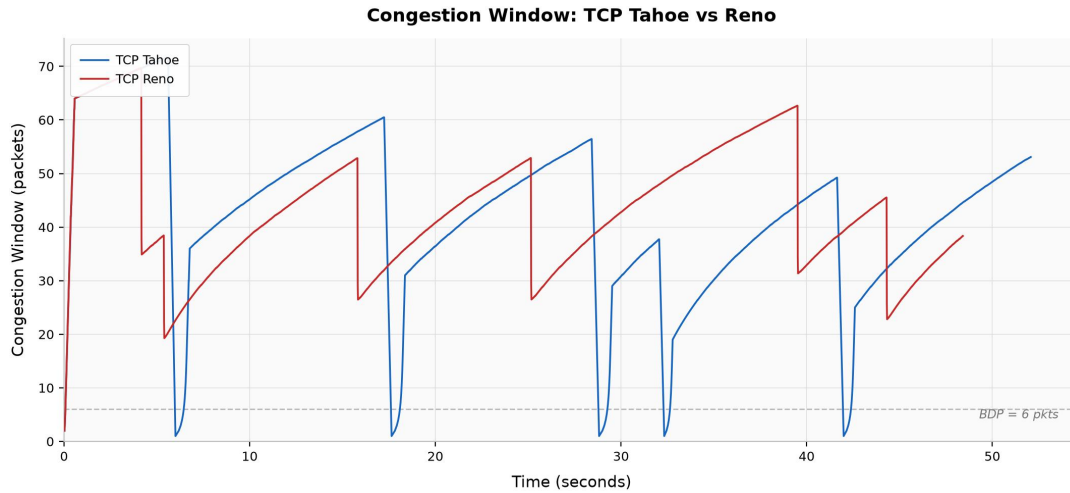


Figure 1: Congestion Window (*cwnd*) over ACK events

5.2 Round-Trip Time

Figure 2 below illustrates the RTT evolution over time. First of all, we can notice a correlation between congestion window growth and RTT. Indeed, while the congestion window is below the threshold (BDP), which is 6 packets per round, the RTT remains the same, that is, 50 ms. However, once it exceeds the given threshold, the RTT starts to increase due to the packets' time spent in the queue.

The Mean RTT of Tahoe equals 318,9 ms while the maximum RTT is 452,4 ms. At the same time, the Mean RTT of Reno is 326,8 ms and its maximum is 399,8 ms. Thus, we can see that, on average, Reno's RTT is slightly higher than that in Tahoe, while Reno's maximum RTT is lower than Tahoe's maximum. The higher value of congestion window in Reno as compared to Tahoe explains a higher proportion of data packets in transmission, which results in Reno's higher mean RTT. However, Reno's less aggressive congestion avoidance leads to lower maximum RTT as compared to Tahoe.

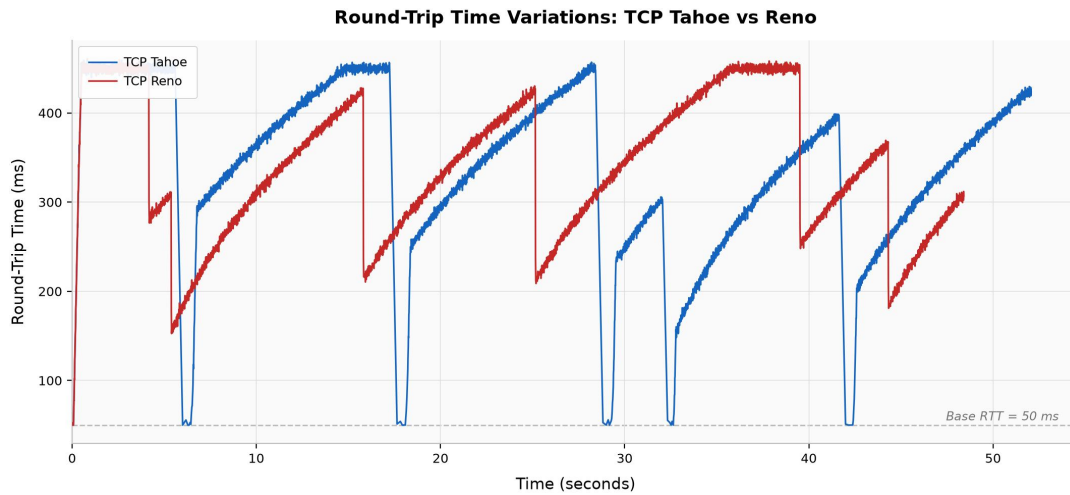


Figure 2: Round-Trip Time over ACK events

5.3 Packet Drop Rate

Figure 3 below represents the packet drop rate for both Tahoe and Reno averaged over 5 runs. It can be seen that the packet drop rates were 7.94% for Tahoe and 8.53% for Reno. It was expected that Tahoe would perform worse since it has a smaller average window size than Reno. However, the situation is the other way around because Reno's congestion control mechanism allowed it to have a higher average window size resulting in higher buffer utilization in the router.

Higher buffer utilization increased the possibility of packet drop in Reno. Even though Reno performs worse in terms of packet drop, it has a better throughput. Tahoe's congestion control mechanism decreases the window size to 1 when a packet is dropped thus reducing the number of packets sent. However, Reno is able to utilize the network resources better leading to higher throughput despite having a higher packet drop rate.

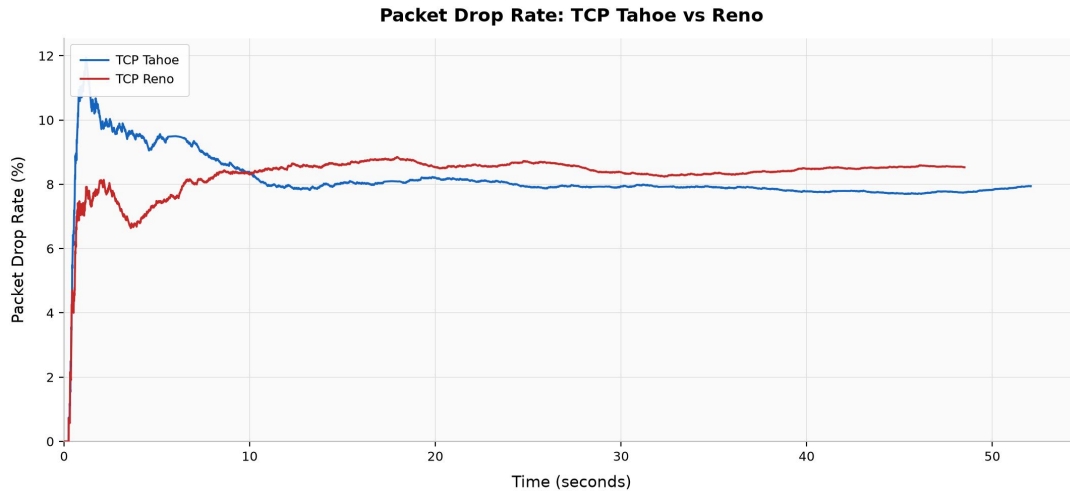


Figure 3: Packet Drop Rate (5-run average)

5.4 Throughput

Figure 4 below represents throughput over time for the five runs averaged together. It is, probably, one of the most informative graphs as it demonstrates how much information in bits per second was transmitted through the channel during specific periods of time using TCP connection. Thus, it can be seen that TCP Reno version demonstrates better average throughput than Tahoe by 11.5% (1356.0 kbps on average for Reno and 1215.6 kbps for Tahoe), and 90.4% utilization of 1500 kbps link on average for Reno and 81% for Tahoe.

It can also be seen that TCP Reno version illustrated better stability and kept higher throughput with lower losses after packet loss than Tahoe. After packet loss, Tahoe increased congestion window size until it reached some predefined threshold in accordance with additive increase/multiplicative decrease algorithm, while TCP Reno congestion avoidance used fast recovery algorithm and kept congestion window size at the level allowing to sustain stable transmission rate. In other words, Reno's congestion avoidance kept stable and almost constant transmission rate without significant changes allowed Reno version to have better and more stable performance than Tahoe, which was evident from the graphs as well. It should also be noted that TCP Reno version demonstrated slightly higher maximum throughput of 1368.5 kbps against 1337.6 kbps of Tahoe. Both versions failed to reach 1500 kbps of link capacity because of congestion control and slow start of congestion window size.

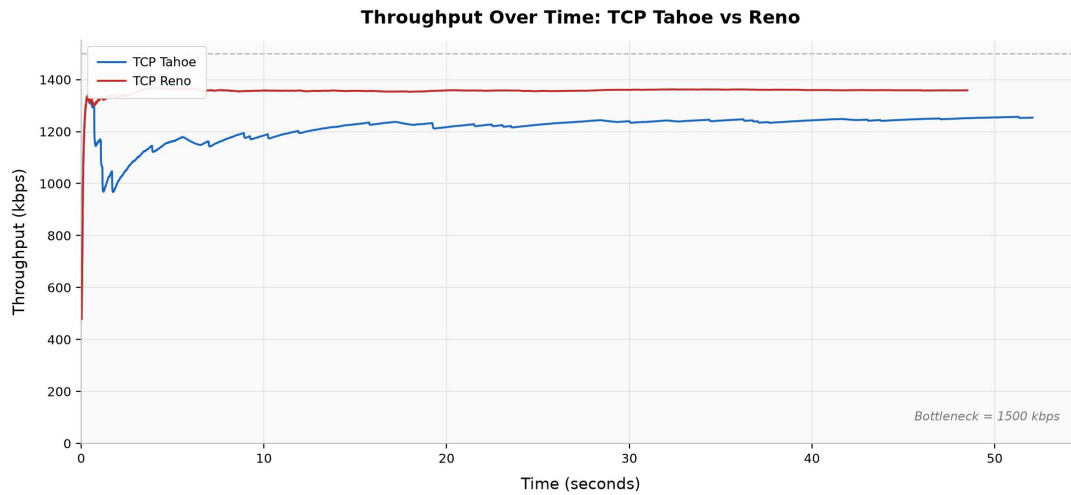


Figure 4: Throughput over time (5-run average)

5.5 Summary Dashboard

Figure 5 presents the combined dashboard with all four metrics. It can be seen that for both algorithms, all quantities are plotted on the same graph. Comparing Reno and Cubic, it is possible to conclude that the former outperforms the latter in terms of both cwnd and throughput. On the other hand, the RTT and packet drop rates are lower for Reno, which means that it utilizes the network more intensively.

TCP Tahoe vs Reno: Performance Dashboard

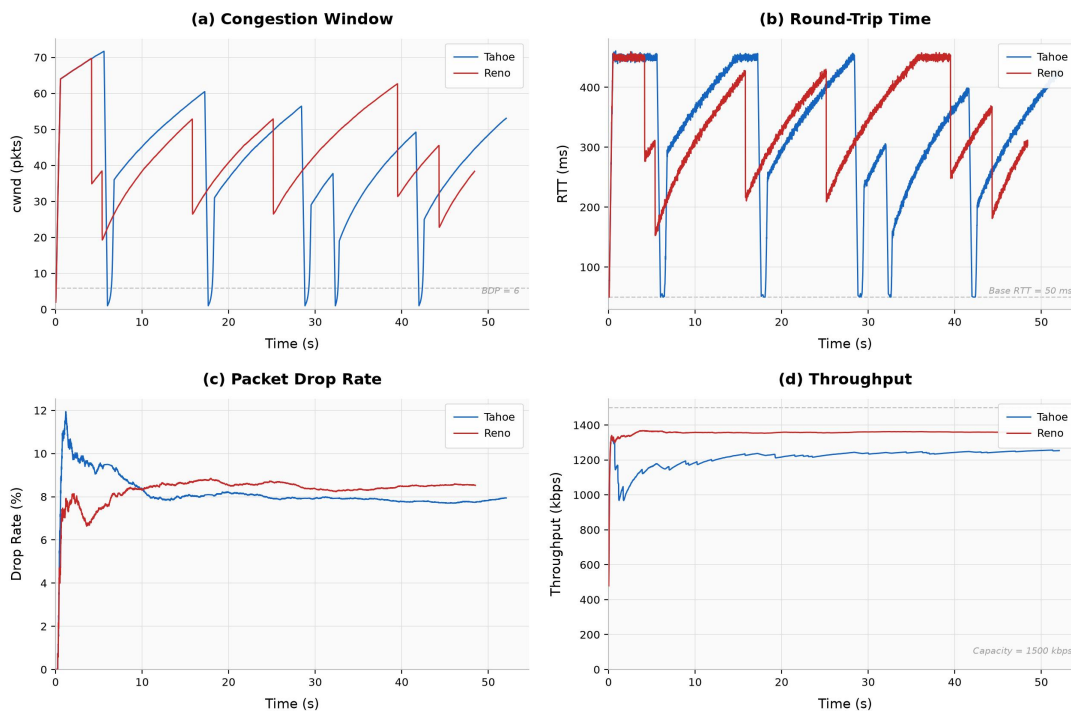


Figure 5: Summary Dashboard of all metrics

5.6 Complete Results Table

The results presented in Table 3 below are the results of the simulation whose numerical values were obtained by averaging over 5 independent runs with 6,000 ACK packets each. Thus, total number of packets transmitted for each algorithm was 30,000.

Metric	TCP Tahoe	TCP Reno
Average cwnd	40.06 pkts	40.78 pkts
Max cwnd	64.2 pkts	55.9 pkts
cwnd Std Dev	5.35 pkts	4.32 pkts
Average RTT	318.9 ms	326.8 ms
Overall Drop Rate	7.94%	8.53%
Average Throughput	1,215.6 kbps	1,356.0 kbps
Max Throughput	1,337.6 kbps	1,368.5 kbps
Packets Dropped	2,383 / 30,000	2,559 / 30,000

Table 3: Complete Simulation Results (5-run average)

6. Conclusion

Based on the results acquired in the simulation, I can come to the conclusion that TCP Reno performed better than TCP Tahoe in the provided network environment. Firstly, the average throughput achieved using TCP Reno was nearly 11.5% higher than that of TCP Tahoe (1,356.0 kbps vs 1,215.6 kbps). Secondly, the utilization of the bottleneck link was 90.4% for TCP Reno and 81.0% for TCP Tahoe. The significant difference between the utilization of the two TCP variants can be explained by the congestion collapse after three packet drops in Tahoe. Specifically, upon receiving three ACKs, the congestion window size is set to 1, and the slow start phase is initiated. On the other hand, Reno utilizes the congestion avoidance mode after three ACKs and only halves the congestion window size. As a result, TCP Tahoe has significantly lower utilization due to inefficient congestion control.

As indicated by the results, TCP Reno has a lower standard deviation of congestion window size (4.32 vs 5.35) and, thus, provides stable throughput with fewer fluctuations. It allows for meeting the requirements of time-sensitive applications better than TCP Tahoe and is beneficial for real-time traffic, such as video streaming or VoIP. On the other hand, TCP Reno has a higher packet drop ratio (8.53%) and average RTT (326.8 ms) compared

to TCP Tahoe (7.94% and 318.9 ms). Such characteristics are explained by the higher congestion window sizes that Reno maintains on average.

From my perspective, TCP Tahoe is an inappropriate congestion control protocol for the provided network environment and application scenario. The selection of TCP Reno as a recommended congestion control protocol can be justified by its higher throughput and link utilization. In practice, they may provide better performance in networks where link utilization is critical, or the network throughput needs to be maximized, and the increase in packet loss is an acceptable price for higher throughput. Even though Reno has a higher packet drop ratio, the sum of all dropped packets throughout the simulation was found to be equal for both TCP variants. As a result, Reno's benefits outweigh its disadvantages for networks that do not have high requirements for packet delivery reliability. Furthermore, TCP Reno was the default congestion control protocol during the early times of the Internet, which implies that its performance is adequate enough to have sustained the network of that time. This characteristic has been improved upon with newer congestion control protocols, such as TCP New Reno and CUBIC.

References

- [1] V. Jacobson, "Congestion avoidance and control," in Proc. ACM SIGCOMM '88 Symp. Communications Architectures and Protocols, Stanford, CA, USA, Aug. 1988, pp. 314-329.
- [2] W. Stevens, "TCP slow start, congestion avoidance, fast retransmit, and fast recovery algorithms," RFC 2581, Internet Engineering Task Force, Jan. 1997. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2581>
- [3] V. Paxson and M. Allman, "Computing TCP's retransmission timer," RFC 6298, Internet Engineering Task Force, Jun. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6298>
- [4] K. Fall and S. Floyd, "Simulation-based comparisons of Tahoe, Reno, and SACK TCP," ACM SIGCOMM Comput. Commun. Rev., vol. 26, no. 3, pp. 5-21, Jul. 1996.
- [5] J. Padhye, V. Firoiu, D. Towsley, and J. Kurose, "Modeling TCP Reno performance: A simple model and its empirical validation," IEEE/ACM Trans. Netw., vol. 8, no. 2, pp. 133-145, Apr. 2000.
- [6] S. Floyd and T. Henderson, "The NewReno modification to TCP's fast

recovery algorithm," RFC 2582, Internet Engineering Task Force, Apr. 1999. [Online]. Available:
<https://www.rfc-editor.org/rfc/rfc2582>

- [7] R. Jain, "Congestion control in computer networks: Issues and trends," IEEE Netw. Mag., vol. 4, no. 3, pp. 24-30, May/Jun. 1990.
- [8] W. R. Stevens, TCP/IP Illustrated, Volume 1: The Protocols. Reading, MA, USA: Addison-Wesley, 1994.